Santiago Macias Fernandez
0271617

Mulitmedia and Computer Graphics
26/02/2026

Technical documentation
JAVA IMAGE PROCESSOR

1. Project Overview

This project is a console-based Java application that allows a user to perform multiple iterative image operations on a selected rectangular region of a loaded image. Operations are applied in sequence to an in-memory working image, the final result is saved only once, after the user has finished all modifications.

Core capabilities:

- Load an image from disk into a BufferedImage.
- Define an arbitrary rectangular region via console input.
- Apply one or more operations to that region: Crop, Negative colors, or Rotation.
- Optionally reuse the last region or define a new one per operation.
- After a Crop, the cropped area becomes the new working image.
- Save the accumulated result as a PNG file at the end.

2. Architecture

The project follows an Object-Oriented design. Responsibilities are divided into five classes and one interface, each with a single, well-defined purpose.

Class Diagram (summary)

Class / Interface	Type	Responsibility
Main	class	Entry point. Handles all console I/O and the modification loop.
ImageProcessor	class	Loads the image, holds the mutable working image, delegates to operations, saves the final result.
Region	class	Encapsulates quadrilateral coordinates, bounding box, barycentric denominators, and the point-in-region test.
ImageOperation	interface	Contract that all operations must fulfil: apply(BufferedImage, Region) into BufferedImage.
CropOperation	class	Implements ImageOperation. Returns a new, smaller BufferedImage containing only the selected region.
NegativeOperation	class	Implements ImageOperation. Inverts RGB channels of every pixel inside the region in-place.
RotateOperation	class	Implements ImageOperation. Extracts the region, rotates it in 90, 180, 270 degrees, and pastes it back centred on the original position.

Design Decisions

Interface as contract. ImageOperation declares a single method. Main and ImageProcessor depend on the interface, not on concrete classes. Adding a new operation requires only creating a new class that implements the interface.

Return value instead of direct file I/O. apply() returns a BufferedImage rather than saving to disk. This allows operations to be chained without creating intermediate files. Saving is deferred to ImageProcessor.saveResult(), called once at the end.

Mutable working image in ImageProcessor. The field currentImg is reassigned when a CropOperation returns a smaller image. NegativeOperation and RotateOperation modify the image in-place and return the same reference, so currentImg is not replaced.

Region invalidation after Crop. When a Crop is performed, the working image dimensions change. Main resets lastRegion to null so the next operation forces the user to enter new valid coordinates relative to the cropped image.

3. Class Reference

Region

Represents the rectangular area the user selects. Although the input is a standard axis-aligned rectangle, the region is internally modelled as a quadrilateral (vertices A, B, C, D) and tested with barycentric coordinates, which makes the design extensible to arbitrary quadrilaterals in the future.

Constructor

```
Region(BufferedImage img, int xLeft, int yTop, int widthRect, int heightRect)
```

Computes the four vertices, clamps the bounding box to the image dimensions, and pre-calculates the two barycentric denominators denom1 and denom2 used by every call to isPointInside.

Key method

```
boolean isPointInside(int x, int y)
```

Returns true if the pixel at (x, y) lies inside the quadrilateral. Internally splits the quad into two triangles (A-B-C and A-C-D) and applies the barycentric test to each. If the point is in either triangle it is considered inside.

Getters

Method	Returns
`getMinX() / getMaxX()`	Horizontal bounds of the bounding box (clamped to image width).
`getMinY() / getMaxY()`	Vertical bounds of the bounding box (clamped to image height).

`getCropWidth()`	maxX - minX + 1. The +1 ensures the inclusive pixel range is fully covered.
`getCropHeight()`	maxY - minY + 1.

ImageProcessor

Owns the lifecycle of the working image: load, apply, save. It does not contain any pixel logic; it delegates entirely to ImageOperation implementations.

Method	Description
`ImageProcessor(String filePath)`	Loads the image. Throws RuntimeException if the file is not found, IOException if reading fails.
`getImage()`	Returns the current working image. Used by Main to build a Region with the correct dimensions after a Crop.
`process(ImageOperation, Region)`	Calls operation.apply(currentImg, region) and assigns the return value back to currentImg.
`saveResult(String outputPath)`	Writes currentImg to disk as PNG. Called exactly once, after all modifications.

ImageOperation (interface)

```
BufferedImage apply(BufferedImage img, Region region) throws IOException
```

Single-method interface. All three operations implement it. The return value contract is:

- CropOperation → returns a new BufferedImage (smaller, becomes the new working image).
- NegativeOperation → modifies img in-place, returns the same reference.
- RotateOperation → modifies img in-place, returns the same reference.

CropOperation

Scans every pixel in the bounding box of the region. For each pixel that passes the isPointInside test it copies the ARGB value to the corresponding relative position in a new TYPE_INT_ARGB BufferedImage. Pixels outside the region remain transparent (alpha = 0). The new image is returned as the working image; coordinates are now relative to the top-left of the cropped area.

Side effect on Region. After a Crop, Main sets lastRegion = null because the working image now has different dimensions and previous absolute coordinates are invalid.

NegativeOperation

For each pixel inside the region, reads the packed ARGB integer, constructs a Color object, and computes the negative by subtracting each channel from 255:

```
R' = 255 - R     G' = 255 - G     B' = 255 - B
```

The alpha channel is not modified. The pixel is written back to the same coordinates in the original image. No new BufferedImage is created.

RotateOperation

Constructor receives the rotation angle (90, 180, or 270 degrees). The operation proceeds in four phases:

- Extract: copies the region pixels into a temporary sourcePatch image using relative coordinates.
- Allocate rotated buffer: for 90° and 270° the dimensions are swapped (height becomes width and vice versa); for 180° they stay the same.
- Rotate: maps each non-transparent source pixel to its rotated destination coordinate using the standard 2D rotation formulas for multiples of 90°.
- Paste back: clears the original region to white, calculates the centre of the original bounding box, and pastes the rotated patch centred on that point. Pixels that would fall outside the image bounds are discarded.

Angle	Width of rotatedPatch	Pixel mapping
90°	cropHeight	[cropH-1-y, x]
180°	cropWidth	[cropW-1-x, cropH-1-y]
270°	cropHeight	[y, cropW-1-x]

4. Image Changes

The table below shows how currentImg in ImageProcessor changes depending on the operation sequence chosen.

Operation applied	currentImg after	Notes
Negative	Same reference	Pixels modified in-place. Image dimensions unchanged.
Rotate	Same reference	Region cleared to white, rotated patch pasted back. Dimensions unchanged.
Crop	New reference	New smaller BufferedImage. Previous coordinates are invalidated. lastRegion set to null.

5. Key Algorithms

Bounding Box Clamping

To avoid iterating over the entire image, Region computes the smallest axis-aligned rectangle that encloses the quadrilateral. The coordinates are clamped to [0, imageWidth-1] and [0, imageHeight-1] to handle regions that partially exceed the image bounds.

Barycentric Point in Triangle Test

A pixel (x, y) is considered inside the quadrilateral if it lies inside at least one of the two triangles that partition it (A-B-C and A-C-D). Each triangle is tested using barycentric coordinates:

```
D1 = ((yB-yC)(x-xC) + (xC-xB)(y-yC)) / denom1
D2 = ((yC-yA)(x-xC) + (xA-xC)(y-yC)) / denom1
D3 = 1 - D1 - D2
```

If D1, D2, and D3 are all >= 0, the point is inside the triangle. The denominators are pre-computed once in the Region constructor and reused for every pixel to avoid redundant arithmetic.

Rotation Mapping

For a source pixel at (x, y) in a patch of size cropWidth x cropHeight, the destination coordinates in the rotated buffer are:

```
90°:   dest = (cropHeight - 1 - y,   x)
180°:  dest = (cropWidth - 1 - x,    cropHeight - 1 - y)
270°:  dest = (y,                    cropWidth - 1 - x)
```

After rotation, the patch is re-centred on the original region's centroid before being pasted back onto the working image.

6. File Structure

File	Description
`Main.java`	Entry point. Console UI and modification loop.
`ImageProcessor.java`	Manages the working image lifecycle.
`Region.java`	Quadrilateral region, bounding box, and point-in-region logic.
`ImageOperation.java`	Interface defining the operation contract.
`CropOperation.java`	Crops the working image to the selected region.

`NegativeOperation.java`	Applies color inversion to the region.
`RotateOperation.java`	Rotates region contents 90, 180, or 270 degrees.
`src/name.extension`	Default input image (path configured in Main) (can be changed).
`output_edited_image.png`	Output file written after all modifications are complete.

7. Error Handling

Situation	Handling
Image file not found	ImageProcessor constructor throws RuntimeException. Main catches IOException and prints the error.
Invalid menu option	Default branch prints a message; no operation is applied. The loop continues.
Invalid rotation value	Inner while loop repeats until the user enters 90, 180, or 270. Non-integer input is consumed and discarded via scanner.next().
Region outside image bounds	Bounding box is clamped in Region constructor. Pixels outside the image are never accessed.
Rotated patch exceeds image bounds	RotateOperation validates each target coordinate before calling setRGB.